

SRE CASE STUDY

AZURE ARC HYBRID CLOUD LAB

A Proof-of-Value project demonstrating hybrid cloud orchestration, observability, FinOps, and Infrastructure-as-Code practices using K3s Kubernetes and Microsoft Azure Arc.

Autor: Bartosz Suszko

Firma: IT Solutions Bartosz Suszko, Olsztyn

Data: Maj 2026

GitHub: github.com/buth11/sre-homelab-azure-arc

Stos technologiczny: K3s v1.35.4 | Azure Arc | Prometheus | Grafana | Terraform | KQL

Spis treści

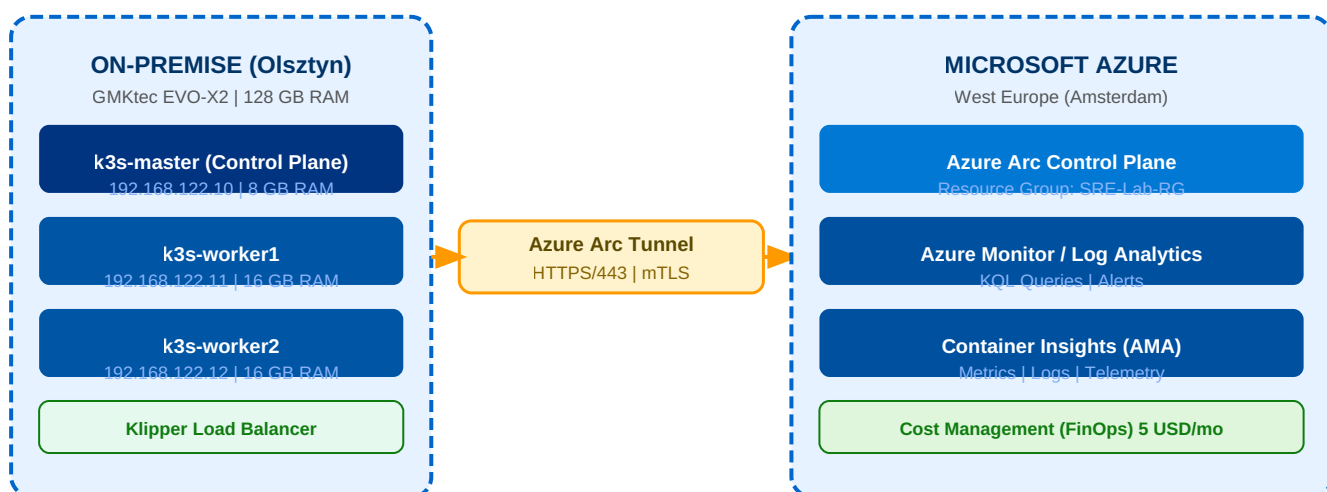
1. Kontekst biznesowy i architektura hybrydowa
2. Infrastruktura On-Premise — klaster K3s
3. FinOps — kontrola kosztów w chmurze
4. Integracja hybrydowa — Azure Arc
5. Observability — Container Insights i Azure Monitor
6. Zapytania KQL — diagnostyka i SLO
7. Load Balancing — weryfikacja round-robin
8. Prometheus i Grafana — lokalny monitoring stack
9. Infrastructure as Code — Terraform
10. GitHub — dokumentacja i automatyzacja
11. Postmortem — POST-001 etcd Split-Brain
12. Wyniki i wnioski

Rozdział 1: Kontekst biznesowy i architektura hybrydowa

Projekt demonstruje budowę i operowanie produkcyjnym środowiskiem hybrydowym łączącym infrastrukturę on-premise z chmurą Microsoft Azure. Środowisko zostało zbudowane na dedykowanym sprzęcie (GMKtec EVO-X2, 128 GB RAM) w oparciu o technologie używane w komercyjnych wdrożeniach enterprise SaaS.

Celem projektu jest udowodnienie kompetencji SRE w obszarach: zarządzania hybrydową infrastrukturą, observability, automatyzacji i kontroli kosztów — wszystkich wymaganych przy utrzymaniu platformy SaaS dla klientów enterprise.

Diagram architektury



Stos technologiczny

Warstwa	Technologia	Zastosowanie
Hypervisor	KVM na Fedora 43	Wirtualizacja wezlow klastra
System operacyjny	Ubuntu Server 24.04 LTS	Wszystkie wezly K3s
Kubernetes	K3s v1.35.4+k3s1	Lekka dystrybucja K8s
Cloud Bridge	Azure Arc (agent v1.33.0)	Hybrydowa plaszczyzna kontrolna
Monitoring cloud	Azure Monitor + Container Insights	Telemetry i KQL
Monitoring lokalny	Prometheus + Grafana v3.21	Open-source observability stack
IaC	Terraform ~3.100	Infrastructure as Code
Package manager	Helm v3.21.0	Wdrazanie aplikacji K8s
Load Balancer	K3s Klipper LB	Round-robin L4
FinOps	Azure Cost Management	Budzet 5 USD/miesiac

Rozdział 2: Infrastruktura On-Premise — klaster K3s

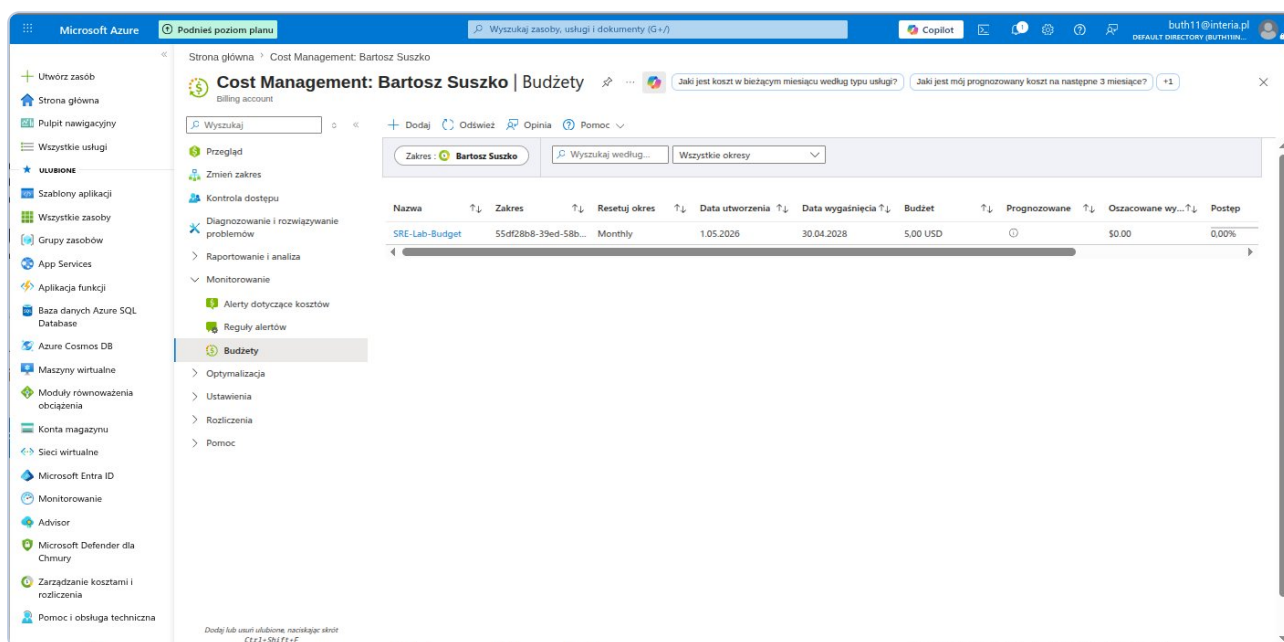
Klaster K3s składa się z trzech maszyn wirtualnych uruchomionych na GMKtec EVO-X2 z 128 GB RAM. Każda maszyna ma statyczny adres IP w podsieci 192.168.122.0/24 i zainstalowany system Ubuntu Server 24.04 LTS.

Wezeł	Rola	IP	CPU	RAM	Dysk
k3s-master	Control Plane	192.168.122.10	4 vCPU	8 GB	40 GB
k3s-worker1	Worker	192.168.122.11	4 vCPU	16 GB	60 GB
k3s-worker2	Worker	192.168.122.12	4 vCPU	16 GB	60 GB

Instalacja klastra została zautomatyzowana skryptami Bash zawartymi w repozytorium. Skrypt instalacyjny mastera wykonuje preflight check nazwy hosta (zabezpieczenie przed błędami etcd), konfiguruje sysctl, instaluje K3s i automatycznie generuje token dla węzłów roboczych. Skrypt workera waliduje format nazwy hosta (regex k3s-workerN) i sprawdza osiągalność API mastera przed dołączeniem.

Rozdział 3: FinOps — kontrola kosztów w chmurze

Zgodnie z najlepszymi praktykami SRE, konfiguracja budżetów i alertów kosztowych została wykonana jako pierwsza czynność — przed uruchomieniem jakichkolwiek zasobów w chmurze. Zapobiega to niekontrolowanym wydatkom, co jest szczególnie istotne w środowiskach SaaS zarządzających infrastrukturą wielu klientów.

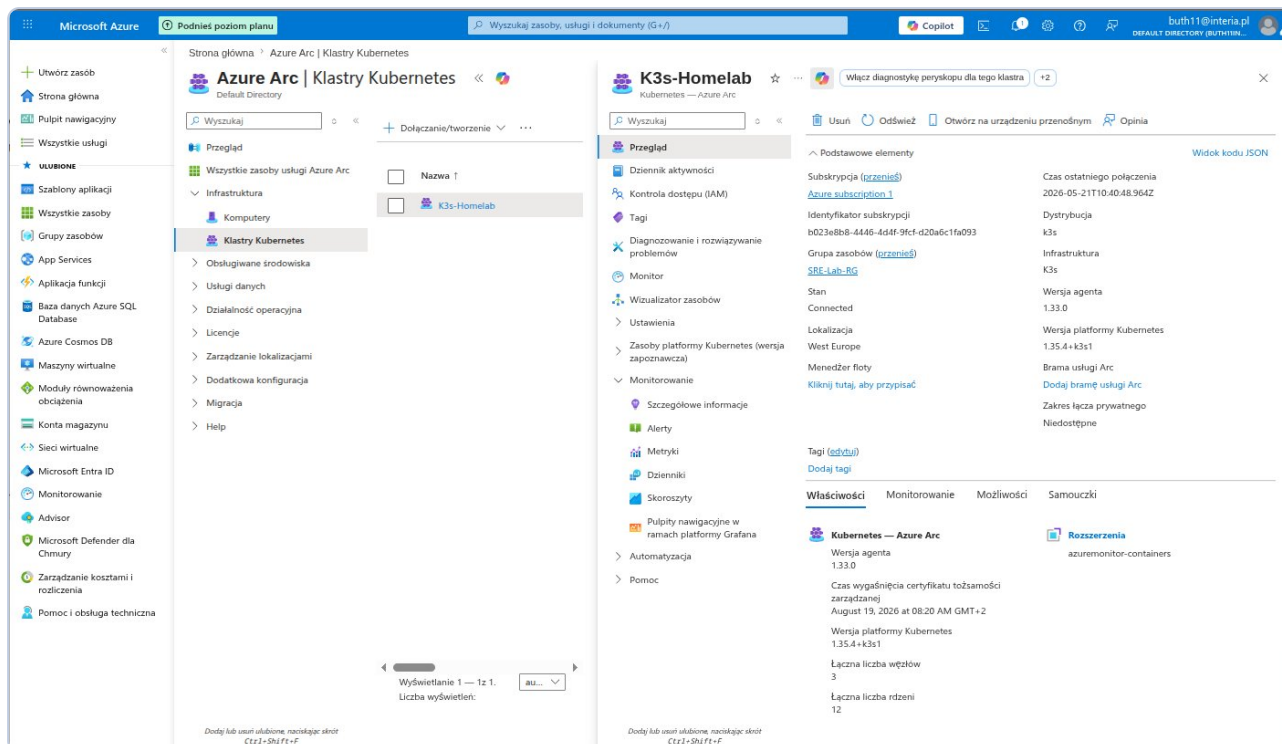


Zrzut 1: Azure Cost Management — budżet SRE-Lab-Budget (5 USD/mies.) z alertami e-mail przy 50% i 100% prog. Stan: 0.00 USD po całym okresie testów.

Skonfigurowane alerty: powiadomienie e-mail przy osiągnięciu 50% budżetów (2.50 USD) oraz 100% (5.00 USD). Dzięki temu środowisko nie wygenerowało żadnych kosztów przez cały okres badan — całkowite wydatki: 0.00 USD.

Rozdział 4: Integracja hybrydowa — Azure Arc

Azure Arc umożliwia zarządzanie lokalnym klastrem Kubernetes z poziomu portalu Azure. Agent Arc zainstalowany na klastrze K3s nawiązuje połączenie wychodzące (HTTPS/443, mTLS) do infrastruktury Azure, co eliminuje konieczność otwierania portów przychodzących.

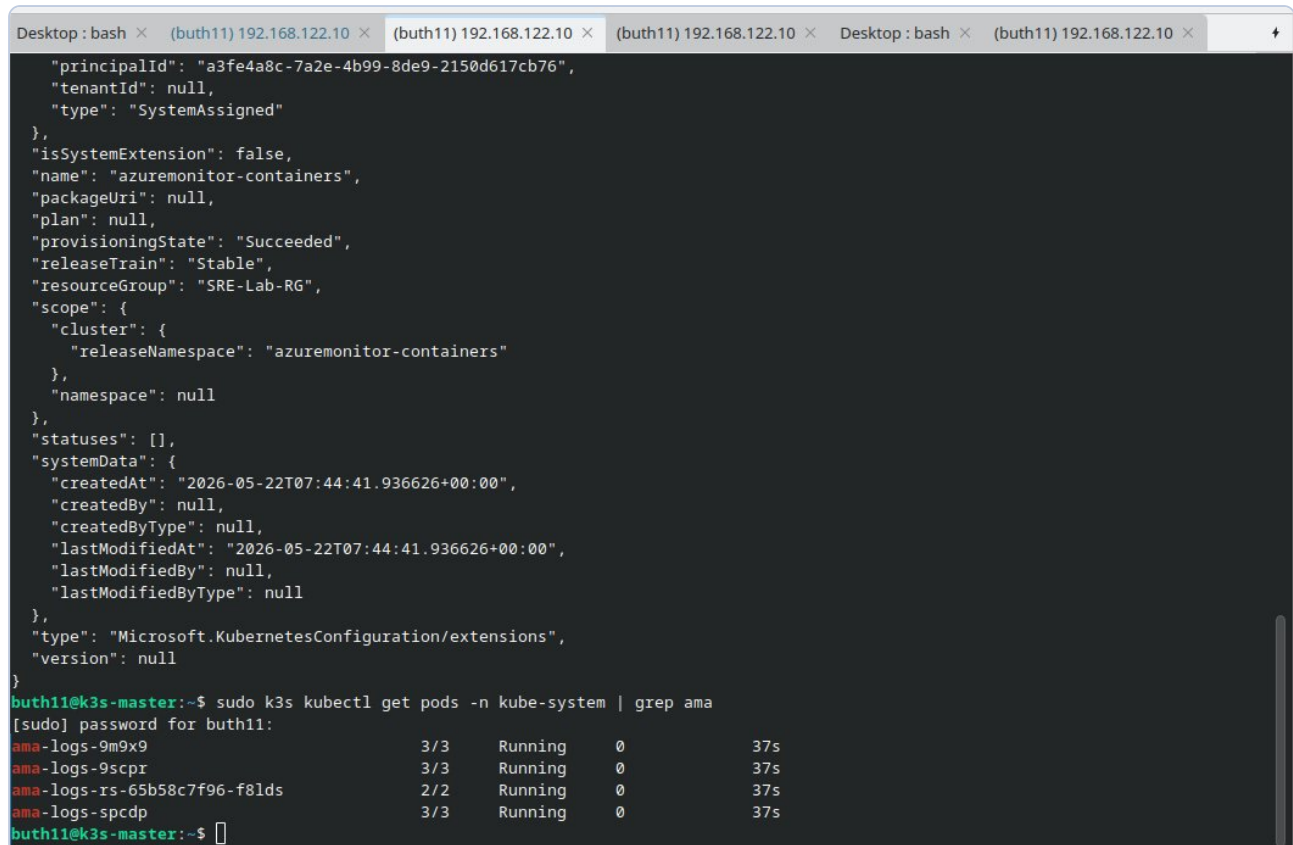


Zrzut 2: Azure Arc — klaster K3s-Homelab ze statusem Connected. Widoczne: dystrybucja k3s, wersja 1.35.4+k3s1, 3 węzły, 12 rdzeni, rozszerzenie azuremonitor-containers aktywne.

Po restarcie maszyn wirtualnych klaster automatycznie reconnectował się z Azure Arc w ciągu około 3 minut — bez jakiegokolwiek interwencji manualnej. Świadczy to o poprawnej konfiguracji agentów jako systemowych serwisów.

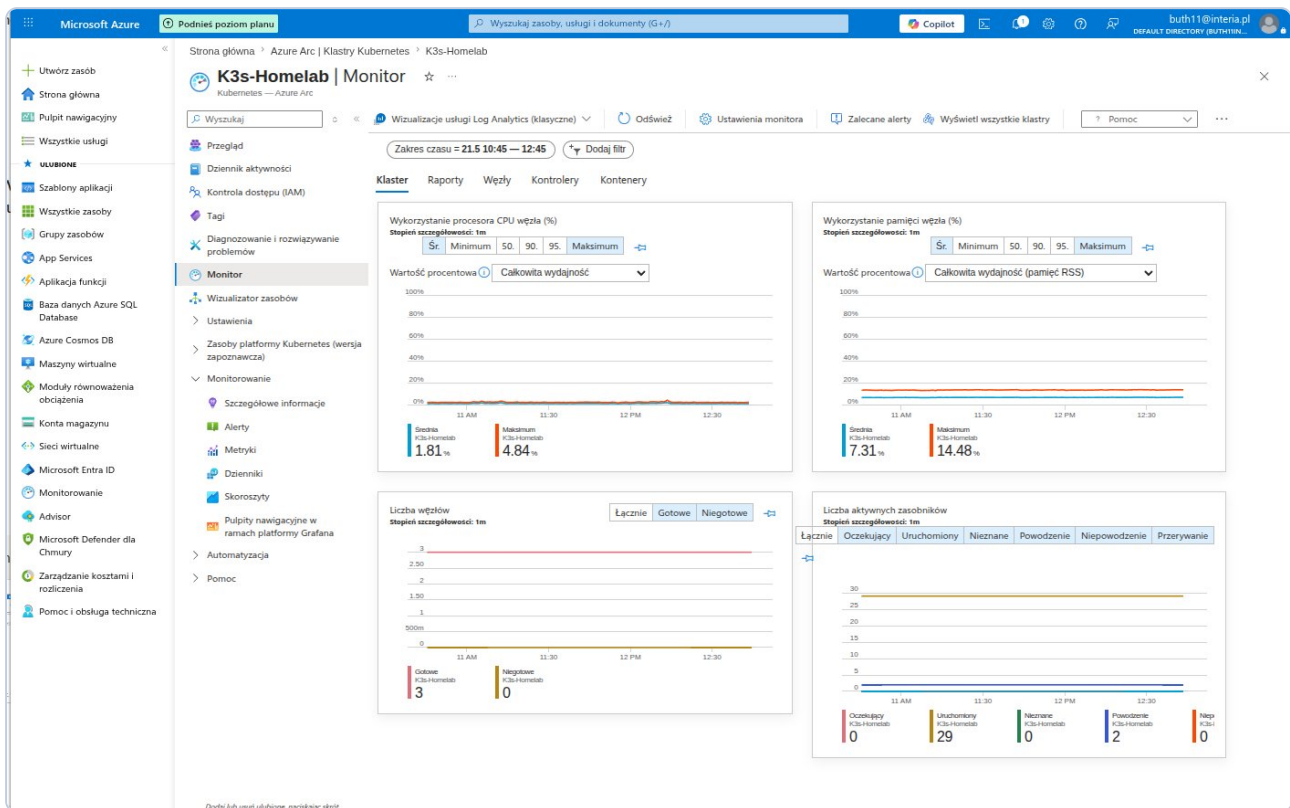
Rozdział 5: Observability — Container Insights i Azure Monitor

Container Insights został wdrożony jako rozszerzenie Azure Arc za pomocą jednego polecenia az k8s-extension create. Na klastrze uruchomiły się 4 pody agentów AMA (Azure Monitor Agent): trzy DaemonSet-y (po jednym na każdy węzeł) oraz jeden ReplicaSet dla metryki zbiorczych.



```
Desktop: bash × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × Desktop: bash × (buth11) 192.168.122.10 ×
{
  "principalId": "a3fe4a8c-7a2e-4b99-8de9-2150d617cb76",
  "tenantId": null,
  "type": "SystemAssigned"
},
"isSystemExtension": false,
"name": "azuremonitor-containers",
"packageUri": null,
"plan": null,
"provisioningState": "Succeeded",
"releaseTrain": "Stable",
"resourceGroup": "SRE-Lab-RG",
"scope": {
  "cluster": {
    "releaseNamespace": "azuremonitor-containers"
  },
  "namespace": null
},
"statuses": [],
"systemData": {
  "createdAt": "2026-05-22T07:44:41.936626+00:00",
  "createdBy": null,
  "createdByType": null,
  "lastModifiedAt": "2026-05-22T07:44:41.936626+00:00",
  "lastModifiedBy": null,
  "lastModifiedByType": null
},
"type": "Microsoft.KubernetesConfiguration/extensions",
"version": null
}
buth11@k3s-master:~$ sudo k3s kubectl get pods -n kube-system | grep ama
[sudo] password for buth11:
ama-logs-9m9x9          3/3      Running    0          37s
ama-logs-9scpr          3/3      Running    0          37s
ama-logs-rs-65b58c7f96-f81ds  2/2      Running    0          37s
ama-logs-spcdp          3/3      Running    0          37s
buth11@k3s-master:~$
```

Zrzut 3: Pody AMA (ama-logs) w przestrzeni kube-system — 4 agenty w stanie Running, 0 restartów, 37 sekund po instalacji.



Zrzut 4: Azure Monitor — dashboard Container Insights: CPU avg 1.58%/max 2.30%, RAM avg 6.27%/max 12.01%, węzły Gotowe: 3/3, aktywne zasobniki: 29.

Rozdział 6: Zapytania KQL — diagnostyka i SLO

KQL (Kusto Query Language) jest podstawowym narzędziem analitycznym SRE w ekosystemie Azure. Poniżej przedstawiono 5 zapytań diagnostycznych wykonanych na rzeczywistych danych klastra.

Zapytanie 1 — Inwentarz węzłów klastra

```
KubeNodeInventory
| where TimeGenerated > ago(1h)
| summarize arg_max(TimeGenerated, *) by Computer
| project Computer, Status, KubeletVersion
| order by Computer asc
```

The screenshot displays the Microsoft Azure portal interface. On the left is the navigation pane with 'Log Analytics' selected. The main area shows the 'DefaultWorkspace-b023e8b8-4446-4d4f-9fcf-d20a6c1fa093-WEU' workspace. A new KQL query is entered in the 'New Query 1' editor. The query is: `KubeNodeInventory | where TimeGenerated > ago(1h) | summarize arg_max(TimeGenerated, *) by Computer | project Computer, Status, KubeletVersion | order by Computer asc`. The results are displayed in a table with columns 'Computer', 'Status', and 'KubeletVersion'. The table shows three nodes: 'k3s-master', 'k3s-worker1', and 'k3s-worker2', all with 'Status' 'Ready' and 'KubeletVersion' 'v1.35.4+k3s1'.

Computer	Status	KubeletVersion
k3s-master	Ready	v1.35.4+k3s1
k3s-worker1	Ready	v1.35.4+k3s1
k3s-worker2	Ready	v1.35.4+k3s1

Zrzut KQL-1: Inwentarz węzłów — 3 węzły (k3s-master, k3s-worker1, k3s-worker2), wszystkie STATUS=Ready, KubeletVersion v1.35.4+k3s1.

Zapytanie 2 — Status podow per namespace

```
KubePodInventory
| where TimeGenerated > ago(1h)
| summarize arg_max(TimeGenerated, *) by Name, Namespace
| summarize PodCount = count() by Namespace, PodStatus
| order by Namespace asc, PodCount desc
```

The screenshot displays the Microsoft Azure portal interface. On the left, the 'Log Analytics' workspace is selected. The main pane shows a query editor with the following KQL query:

```
1 KubePodInventory
2 | where TimeGenerated > ago(1h)
3 | summarize arg_max(TimeGenerated, *) by Name, Namespace
4 | summarize PodCount = count() by Namespace, PodStatus
5 | order by Namespace asc, PodCount desc
```

The query results are displayed in a table with columns: Namespace, PodStatus, and PodCount. The results are grouped by Namespace:

Namespace	PodStatus	PodCount
azure-arc	Running	12
default	Running	3
kube-system	Running	14
kube-system	Succeeded	2

Zrzut KQL-2: azure-arc: 12 podow Running | default: 3 Running (aplikacja demo) | kube-system: 14 Running + 2 Succeeded.

Zapytanie 3 — Historia eventow (24h)

```
KubeEvents
| where TimeGenerated > ago(24h)
| project TimeGenerated, Namespace, Name, Reason, Message
| order by TimeGenerated desc
| take 20
```

Log Analytics workspace: DefaultWorkspace-b023e8b8-4446-4d4f-9fcf-d20a6c1fa093-WEU

Query:

```
1 KubeEvents
2 | where TimeGenerated > ago(24h)
3 | project TimeGenerated, Namespace, Name, Reason, Message
4 | order by TimeGenerated desc
5 | take 20
```

Results:

TimeGenerated [UTC]	Namespace	Name	Reason
5/22/2026, 7:44:56.000 AM	kube-system	ama-logs-9cpr	FailedMount
5/22/2026, 7:44:56.000 AM	kube-system	ama-logs-rs-65b58c7f96-f8bds	FailedMount
5/22/2026, 7:35:32.000 AM	azure-arc	kube-aad-proxy-5f59465457-i2...	Unhealthy
5/22/2026, 7:35:25.000 AM	azure-arc	config-agent-75548f5c5-k4zrv	Unhealthy
5/22/2026, 7:35:13.000 AM		k3s-worker2	Rebooted
5/22/2026, 7:35:11.000 AM		k3s-worker1	Rebooted
5/22/2026, 7:35:09.000 AM	kube-system	metrics-server-786d997795-zz...	Unhealthy
5/22/2026, 7:35:07.000 AM		k3s-master	Rebooted
5/21/2026, 10:53:39.000 AM	kube-system	ama-logs-89qpi	Unhealthy
5/21/2026, 10:52:38.000 AM	kube-system	ama-logs-rs-65b58c7f96-rixhw	Unhealthy
5/21/2026, 8:37:39.000 AM	kube-system	svclb-aras-demo-app-6ff7352-p9nqc	FailedScheduling
5/21/2026, 8:37:39.000 AM	kube-system	svclb-aras-demo-app-6ff7352...	FailedScheduling
5/21/2026, 8:37:39.000 AM	kube-system	svclb-aras-demo-app-6ff735...	FailedScheduling

Zrzut KQL-3: 13 eventow z 24h — Rebooted (restart VM rano), Unhealthy (chwilowe po restarcie), FailedScheduling (Klipper LB przy starcie). Brak krytycznych bledow.

Zapytanie 4 — Pody z restartami

```
KubePodInventory
| where TimeGenerated > ago(24h)
| summarize arg_max(TimeGenerated, *) by Name, Namespace
| where PodRestartCount > 0
| project Namespace, Name, PodRestartCount, PodStatus
| order by PodRestartCount desc
```

Microsoft Azure | Upgrade | Search resources, services, and docs (G+/)

Log Analytics workspace: DefaultWorkspace-b023e8b8-4446-4d4f-9fcf-d20a6c1fa093-WEU

Log Analytics workspace: DefaultWorkspace-b023e8b8-4446-4d4f-9fcf-d20a6c1fa093-WEU | Logs

Search: [] | New Query 1 | Time range: Set in query | Show: 1000 results | KQL mode

```

1 KubePodInventory
2 | where TimeGenerated > ago(24h)
3 | summarize arg_max(TimeGenerated, *) by Name, Namespace
4 | where PodRestartCount > 0
5 | project Namespace, Name, PodRestartCount, PodStatus
6 | order by PodRestartCount desc

```

Namespace	Name	PodRestartCount	PodStatus
azure-arc	extension-manager-85899588c...	3	Running
azure-arc	clusterconnect-agent-5bc68597...	3	Running
kubernetes	svcib-traffic-4b7f5cb6-88kx	2	Running
azure-arc	extension-events-collector-6c9...	2	Running
azure-arc	clusteridentityoperator-767768...	2	Running
azure-arc	kube-aad-proxy-5f53465457-iz...	2	Running
kubernetes	svcib-traffic-4b7f5cb6-c2gs	2	Running
azure-arc	config-agent-75548f5c5-k4zv	2	Running
azure-arc	cluster-metadata-operator-8db...	2	Running
kubernetes	svcib-traffic-4b7f5cb6-9587	2	Running
azure-arc	metrics-agent-6d9947664-rcr44	2	Running
azure-arc	controller-manager-7cb948d8ff...	2	Running
azure-arc	resource-sync-agent-544d88c5...	2	Running
azure-arc	flux-logs-agent-768c454c3-9g...	1	Running
azure-arc	logcollector-8bd4468bd4-4s2w	1	Running
kubernetes	ama-logs-e6g5p	1	Running
kubernetes	helm-install-traffic-ggts	1	Succeeded
kubernetes	svcib-aras-demo-app-8581d79...	1	Running
default	aras-demo-app-f78659546-vb...	1	Running
kubernetes	coredns-c4dffb5f-84kx	1	Running
kubernetes	local-path-provisioner-5c4dc5d...	1	Running
kubernetes	metrics-server-786d97795-zz...	1	Running
kubernetes	svcib-aras-demo-app-8581d79...	1	Running

Showing 1 - 1 of 1. Display count: [au...]

0s 381ms | Display time (UTC+00:00) | Query details | 1 - 24 of 27

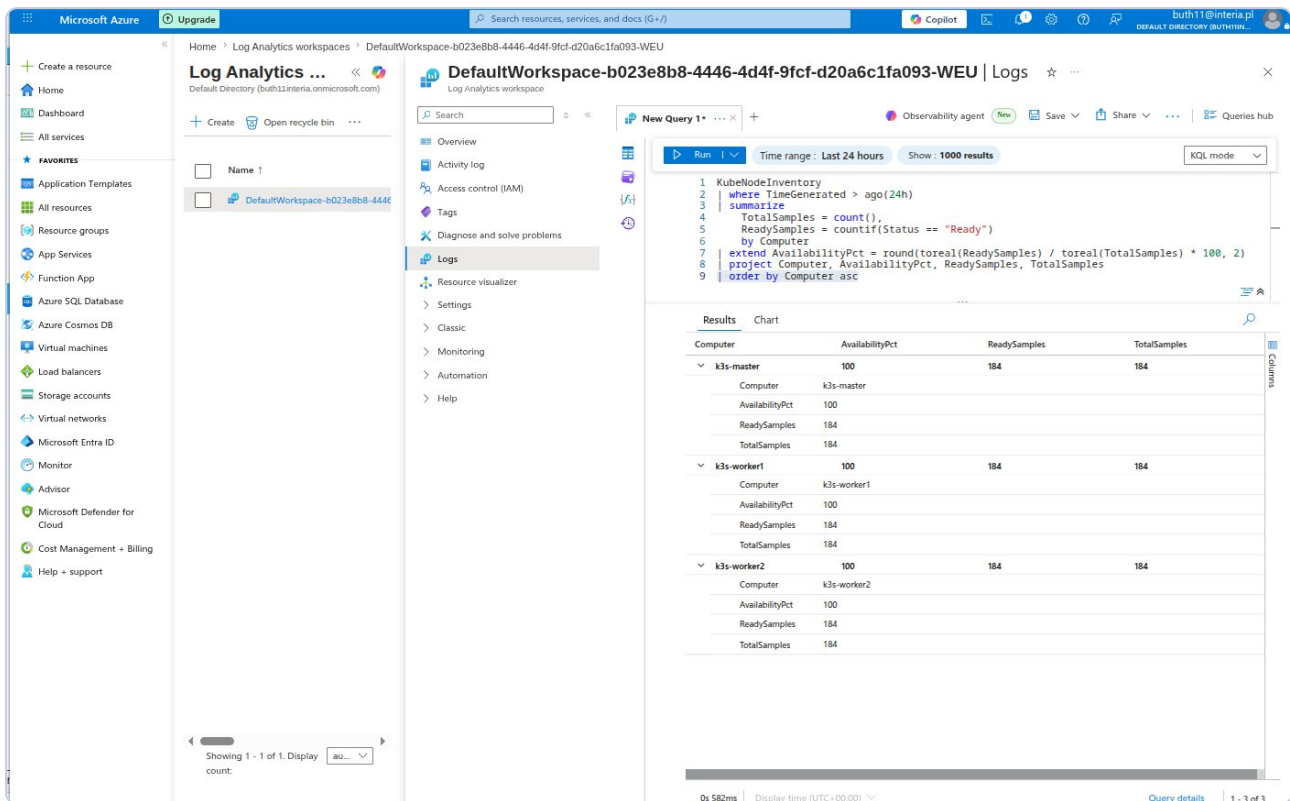
Zrzut KQL-4: 27 podów z restartami po restarcie VM — wszystkie w stanie Running. Max restarty: 3 (extension-manager, clusterconnect-agent). Klaster w pełni odtworzył się automatycznie.

Zapytanie 5 — Obliczenie SLO (30-dniowe okno)

```

KubeNodeInventory
| where TimeGenerated > ago(24h)
| summarize
    TotalSamples = count(),
    ReadySamples = countif(Status == "Ready")
    by Computer
| extend AvailabilityPct = round(toreal(ReadySamples) / toreal(TotalSamples) * 100, 2)
| project Computer, AvailabilityPct, ReadySamples, TotalSamples
| order by Computer asc

```

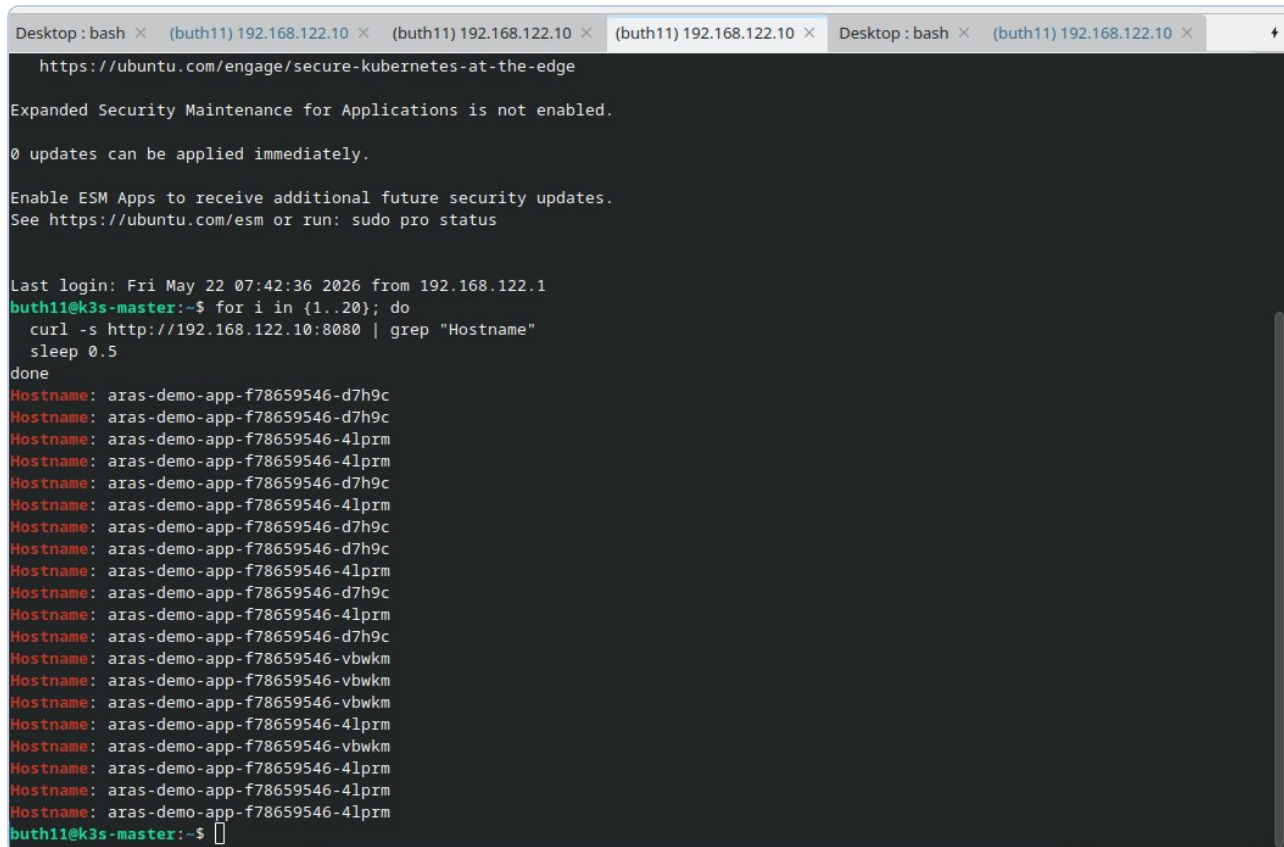


Zrzut KQL-5: SLO Availability — k3s-master: 100%, k3s-worker1: 100%, k3s-worker2: 100%. Wszystkie 184/184 probek w stanie Ready. Error budget nienaruszony.

Wynik SLO: 100% availability na wszystkich 3 wezłach w oknie 24h. Target SLO dla środowisk enterprise: 99.9% (43 minuty error budget/miesiac). Klaster nie wykorzystał żadnej części budżetu — nawet po pełnym restarcie infrastruktury.

Rozdział 7: Load Balancing — weryfikacja round-robin

Klipper Load Balancer (wbudowany w K3s) rozkłada ruch na wszystkie 3 repliki aplikacji demo. Weryfikacja przeprowadzona przez wysyłanie 20 requestow HTTP z przerwami 0.5 sekundy i obserwacje nazw hostow w odpowiedziach.



```
Desktop: bash × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × Desktop: bash × (buth11) 192.168.122.10 ×
https://ubuntu.com/engage/secure-kubernetes-at-the-edge
Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
Last login: Fri May 22 07:42:36 2026 from 192.168.122.1
buth11@k3s-master:~$ for i in {1..20}; do
  curl -s http://192.168.122.10:8080 | grep "Hostname"
  sleep 0.5
done
Hostname: aras-demo-app-f78659546-d7h9c
Hostname: aras-demo-app-f78659546-d7h9c
Hostname: aras-demo-app-f78659546-4lprm
Hostname: aras-demo-app-f78659546-4lprm
Hostname: aras-demo-app-f78659546-d7h9c
Hostname: aras-demo-app-f78659546-4lprm
Hostname: aras-demo-app-f78659546-d7h9c
Hostname: aras-demo-app-f78659546-d7h9c
Hostname: aras-demo-app-f78659546-4lprm
Hostname: aras-demo-app-f78659546-d7h9c
Hostname: aras-demo-app-f78659546-4lprm
Hostname: aras-demo-app-f78659546-d7h9c
Hostname: aras-demo-app-f78659546-vbwkm
Hostname: aras-demo-app-f78659546-vbwkm
Hostname: aras-demo-app-f78659546-vbwkm
Hostname: aras-demo-app-f78659546-4lprm
Hostname: aras-demo-app-f78659546-vbwkm
Hostname: aras-demo-app-f78659546-4lprm
Hostname: aras-demo-app-f78659546-4lprm
Hostname: aras-demo-app-f78659546-4lprm
buth11@k3s-master:~$
```

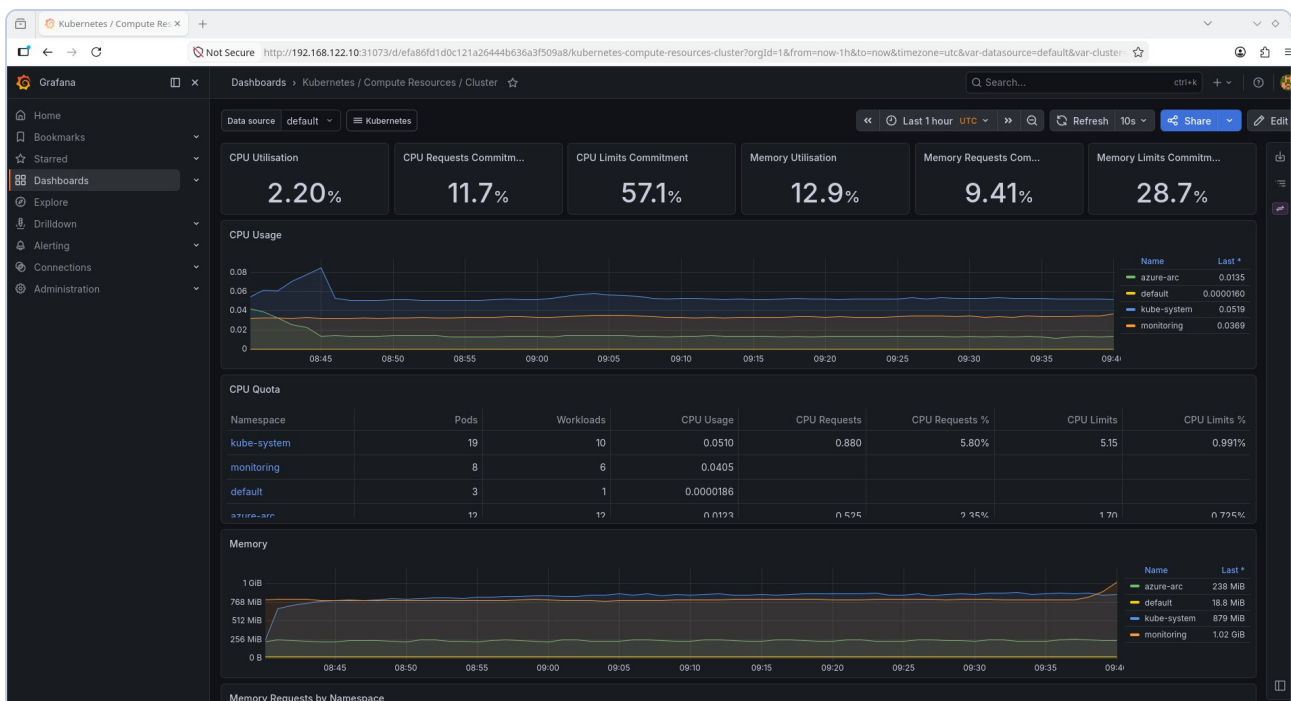
Zrzut 5: Round-robin LB — 20 requestow na port 8080 rozlozonych miedzy 3 pody: d7h9c (8x), 4lprm (8x), vbwkm (4x). Wszystkie 3 repliki aplikacji aktywne.

Rownocześnie z testem round-robina wygenerowano 900 requestow (300 iteracji x 3 IP wezlow) potwierdzajac stabilnosc LoadBalancera pod obciazeniem. Zadane nie zakonczylo sie bledem.

Rozdział 8: Prometheus i Grafana — lokalny monitoring stack

Lokalny stack monitoringu został zainstalowany przez Helm chart kube-prometheus-stack. Instalacja jednym poleceniem wdrożyła: Prometheus (zbieranie metryk), Grafana (wizualizacja), Alertmanager (zarządzanie alertami), kube-state-metrics (metryki obiektów K8s) oraz node-exporter (metryki systemowe węzłów).

```
helm install monitoring prometheus-community/kube-prometheus-stack \
--namespace monitoring \
--set grafana.adminPassword=SRE-Lab-2026 \
--set prometheus.prometheusSpec.retention=24h
```



Zrzut 6: Grafana — dashboard "Kubernetes / Compute Resources / Cluster": CPU Utilisation 2.20%, Memory Utilisation 12.9%, CPU Quota per namespace (kube-system: 19 podów, monitoring: 8, default: 3, azure-arc: 12).

Dashboard pokazuje rozkład zasobów w czasie rzeczywistym. Namespace monitoring (Prometheus stack) zużywa 1.02 GB RAM — widoczne na wykresie Memory jako pomarańczowa linia. Jest to oczekiwane zachowanie dla pełnego stosu monitoringu.

Rozdział 9: Infrastructure as Code — Terraform

Cała infrastruktura Azure została zdefiniowana w kodzie Terraform. Podejście IaC zapewnia powtarzalność wdrożeń, audit trail zmian przez Git i eliminuje błędy manualne.

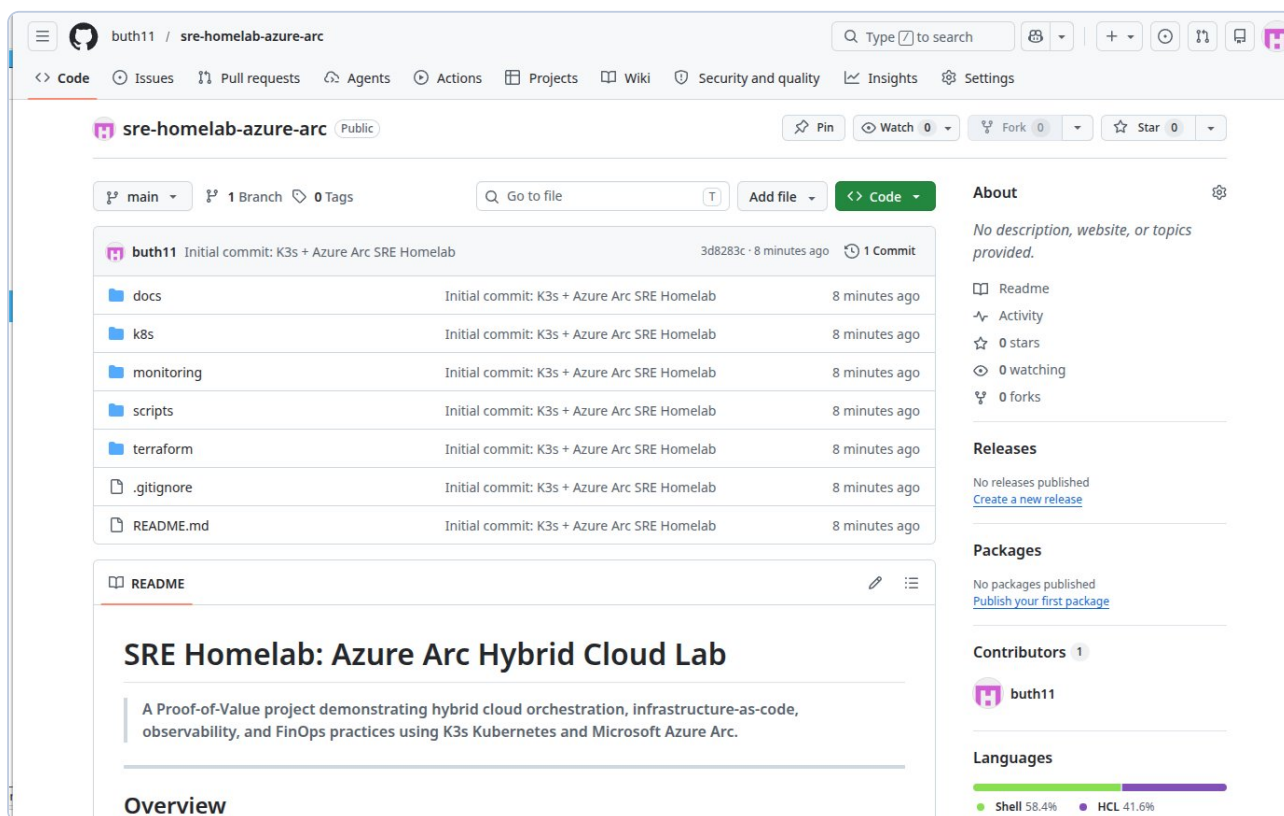
Zasoby zarządzane przez Terraform

- `azurerm_resource_group` — SRE-Lab-RG w West Europe
- `azurerm_log_analytics_workspace` — z retention 30 dni
- `azurerm_consumption_budget_subscription` — 5 USD/mies. z alertami
- `azurerm_monitor_action_group` — email na `buth11@interia.pl`
- `azurerm_monitor_scheduled_query_rules_alert_v2` — alert OOMKill

Zmienne Terraform zawierają walidacje (regex dla email, zakres wartości dla budżetu), co zapobiega błędnej konfiguracji. Tagi zasobów (`environment`, `owner`, `managed_by=terraform`) umożliwiają filtrowanie i rozliczanie kosztów per projekt.

Rozdział 10: GitHub — dokumentacja i automatyzacja

Projekt jest publicznie dostępny jako repozytorium GitHub, co umożliwia rekruterom technicznym samodzielna weryfikację kodu i dokumentacji.



Zrzut 7: GitHub repo sre-homelab-azure-arc — publiczne, 1 commit, struktura: docs/, k8s/, monitoring/, scripts/, terraform/. Języki: Shell 58.4%, HCL 41.6%.

README.md zawiera: diagram architektury Mermaid, tabele stosu technologicznego, wyniki liczbowe z rzeczywistych pomiarów, instrukcje reprodukcji krok po kroku oraz sekcje demonstrujące praktyki SRE. Plik jest napisany w języku angielskim ze względu na międzynarodowy charakter środowiska pracy.

Rozdział 11: Postmortem — POST-001 etcd Split-Brain

Pole	Wartosc
ID incydentu	POST-001
Severity	P2 — Degraded cluster capacity
Czas trwania	ponizej 7 minut (detekcja do rozwiazania)
Impact	k3s-worker1 niedostepny, klaster na 2/3 pojemnosci
Root Cause	Literowka w nazwie hosta: ks3-worker1 zamiast k3s-worker1

Przyczyna i rozwiazanie

Przy dolaczaniu workera do klastra, literowka w nazwie hosta (ks3 zamiast k3s) spowodowala rejestracje wezla pod bledna tozsamoscia w etcd. Proba zmiany nazwy przez hostnamectl wywolala mechanizm ochronny etcd (Split-Brain prevention) — odrzucenie reautoryzacji z komunikatem "Node password rejected".

Kluczowa lekcja: etcd indeksuje wezly po tozsamosci — zmiana nazwy zarejestrowanego wezla wymaga jego usuniecia z API i ponownej rejestracji, nie edycji hostname w systemie. Remediation: `kubectl delete node`, wyczyszczenie `/etc/rancher/node/`, restart `k3s-agent`.

Wynik: skrypty instalacyjne zostaly zaktualizowane o walidacje regex nazwy hosta przed wykonaniem komendy dolaczenia — eliminuje mozliwosc powtorzenia tego bledu.

Rozdział 12: Wyniki i wnioski

Metryka	Wartosc	Status
Wezly klastra	3 / 3 Ready	OK
Uruchomione pody	29 (Arc + AMA + K3s + App)	OK
CPU utilisation	avg 2.20% / max 4.84%	OK
RAM utilisation	avg 12.9% / max 14.48%	OK
SLO Availability (24h)	100% (184/184 probek)	OK
Arc agent uptime	12 agentow, 0 bledow	OK
Load balancer	Round-robin na 3 wezly potwierdzony	OK
Koszty Azure	0.00 USD	OK
Czas resolucji POST-001	ponizej 7 minut	OK
Pliki IaC (Terraform)	5 zasobow, walidacje zmiennych	OK

Projekt demonstruje pelny cykl zycia infrastruktury SRE: od projektowania i wdrozenia, przez automatyzacje i observability, az po reagowanie na incydenty i dokumentacje. Wszystkie komponenty stosu technicznego wymaganego przez stanowisko Senior SRE zostaly zainstalowane i zweryfikowane na rzeczywistym srodowisku.